

ComicCraft – AI Comic Story Creator Using Gemini Models

2. Requirement Analysis Phase

Defining Functional, Interface & Technical Requirements

1. Purpose

This phase converts the ComicCraft concept into practical requirements for development and testing. The listed requirements describe the expected functions, interface behavior, and technical setup of the application.

2. Functional Requirements

- Accept a story prompt, character name, setting, story tone, and selected art style.
- Create a structured five-panel comic outline.
- Expand the outline into narration and character dialogue.
- Generate a comic-style illustration for every panel.
- Associate each illustration with its corresponding panel content.
- Display the generated comic sequentially on a preview page.
- Compile the completed comic into a downloadable PDF.
- Show a confirmation page after successful export.
- Support both form-based and JSON-based generation.
- Provide a separate image-testing route for developers.

3. User Interface Requirements

The homepage should contain clearly labeled inputs and a direct way to begin comic generation. The preview page should display each panel's title, image, scene description, caption, and narration. After export, the success page should confirm completion and allow another comic to be created.

4. Technical Requirements

The project uses Python, FastAPI, Uvicorn, HTML, CSS, Jinja2, Google Gemini APIs, Hugging Face Diffusers, Stable Diffusion, PyTorch, FPDF, Git, and a Python environment manager such as venv. Required packages include fastapi, uvicorn, jinja2, python-multipart, google-generativeai, diffusers, transformers, fpdf, Pillow, and accelerate.

5. Interface and API Inputs

The browser form collects the story prompt, character name, setting, tone, and art style. JSON requests are validated through the Pydantic PromptRequest schema.

6. Outputs and Data

The outline is maintained as a list of panel dictionaries and the story is stored as formatted text. Generated images are saved under static/panels. The layout connects panel numbers, image paths, and text. Exported PDFs are stored under static/exports with timestamped filenames.

7. Operational Requirements

The application should run locally through Uvicorn and be available at 127.0.0.1:8000. FastAPI's interactive documentation is available through /docs for API testing. API keys and tokens should be kept in environment variables instead of being embedded in source code.

8. Scope Notes

The current scope covers single-comic generation and does not include user account management. User profiles, comic libraries, and multi-page story arcs are identified as possible future enhancements.

9. Requirement Analysis Outcome

The requirement analysis establishes an implementation checklist covering input handling, AI generation, image creation, layout assembly, preview rendering, PDF export, API access, and local verification. This provides the foundation for the implementation stage.